

Modül 05: Veri İşleme — dplyr ve tidyr

Prof. Dr. Hakan Mehmetcik

2026-09-11

Veri İşleme İçin Bir Dilbilgisi

Bir araştırmacının zamanının büyük kısmı, aslında “analiz” değil **veri hazırlama**dır: yanlış sütunu atmak, eksik gözlemi ayıklamak, iki tabloyu birleştirmek, yılları sütun yerine satıra çevirmek. Bu iş sıkıcı görünür ama örtük bir tehlike taşır: her elle yapılan düzenleme adımı, tekrarlanamaz ve hatalı olma riski taşır.

dplyr paketi, tıpkı **ggplot2**’nin görselleştirme için bir dilbilgisi sunması gibi, veri manipülasyonu için bir dilbilgisi sunar: küçük sayıda **fiil** (verb), birbirine `|>` ile zincirlenerek karmaşık sorulara dönüşür.

Not

dplyr’ın beş temel fiili

- **select()**: sütun seçer
- **filter()**: satır seçer (koşula göre)
- **mutate()**: yeni sütun türetir
- **arrange()**: satırları sıralar
- **summarize()**: verileri (genellikle gruplar hâlinde) özetler

Bu modülde bu beş fiile, verileri “düzenli” (**tidy**) hâle getiren **tidyr** paketinin iki fonksiyonunu (**pivot_longer()**, **pivot_wider()**) ve birden çok tabloyu birleştiren **dplyr** join fonksiyonlarını ekliyoruz. Üçü birlikte, siyaset bilimi ve kamu yönetiminde karşılaşacağınız neredeyse her veri hazırlama görevini kapsar.

Bu Modülde Kullanacağımız Veri: **gapminder**

Kursun ilk modülünden beri kullandığımız **gapminder** veri setine geri dönüyoruz — ama bu kez odak noktamız görselleştirmek değil, **doğru soruyu sormak için veriyi doğru forma**

getirmek.

```
glimpse(gapminder)
```

```
Rows: 1,704
Columns: 6
$ country   <fct> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ~
$ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, ~
$ year      <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ~
$ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8~
$ pop       <int> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12~
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ~
```

Veri seti, **142 ülke** için **1952–2007** arasında 5 yılda bir ölçülen yaşam beklentisi (**lifeExp**), nüfus (**pop**) ve kişi başı GSYİH (**gdpPercap**) içerir. Bu bir **panel veridir**: hem ülkeler arası (yatay kesit) hem de zaman içi (boylamsal) karşılaştırmaya izin verir — karşılaştırmalı siyasetin klasik veri yapısı.

dplyr Fiilleri

select() — Sütun Seçmek

```
gapminder |> select(country, year, lifeExp) |> head(6)
```

country	year	lifeExp
Afghanistan	1952	28.801
Afghanistan	1957	30.332
Afghanistan	1962	31.997
Afghanistan	1967	34.020
Afghanistan	1972	36.088
Afghanistan	1977	38.438

Belirli sütunları **çıkarmak** için – kullanılır: `select(gapminder, -continent)`. İsim aralığı seçmek için : kullanılabilir: `select(gapminder, country:year)`.

filter() — Satır Seçmek

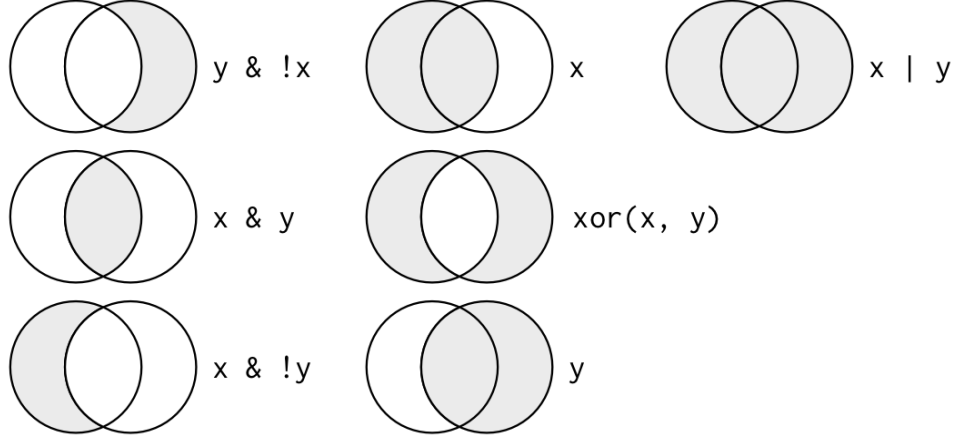
`filter()`, bir veya birden çok mantıksal koşulu sağlayan satırları tutar. Birden çok koşul virgülle ayrıldığında **VE** (&) anlamına gelir.

```
gapminder |>
  filter(continent == "Europe", year == 1997, lifeExp > 75) |>
  select(country, lifeExp) |>
  arrange(desc(lifeExp))
```

country	lifeExp
Sweden	79.390
Switzerland	79.370
Iceland	78.950
Italy	78.820
Spain	78.770
France	78.640
Norway	78.320
Netherlands	78.030
Greece	77.869
Belgium	77.530
Austria	77.510
Germany	77.340
United Kingdom	77.218
Finland	77.130
Ireland	76.122
Denmark	76.110
Portugal	75.970
Montenegro	75.445
Slovenia	75.130

Mantıksal operatörleri hatırlamak için:

```
knitr::include_graphics(here("05_modul", "images", "boolean_operators.png"))
```



Figür 1: Mantıksal operatörlerin Venn şeması

💡 Tavsiye

Sık yapılan hata: `filter(continent == "Europe" | continent == "Asia")` yazacağımıza yanlışlıkla `filter(continent == "Europe" & continent == "Asia")` yazmak — bir ülke aynı anda iki kıtada olamayacağı için ikinci ifade **sıfır satır** döndürür. “Ya biri ya diğeri” için `|` (VEYA), her ikisi birden için `&` (VE) kullanılır.

`arrange()` — Sıralamak

```
gapminder |>
  filter(year == 2007) |>
  arrange(gdpPercap) |>
  select(country, gdpPercap) |>
  head(5)
```

country	gdpPercap
Congo, Dem. Rep.	277.5519
Liberia	414.5073
Burundi	430.0707
Zimbabwe	469.7093
Guinea-Bissau	579.2317

Azalan sırada sıralamak için `desc()`: `arrange(desc(gdpPercap))`.

mutate() — Yeni Sütun Türetmek

gdpPercap, kişi başına GSYİH'dir; toplam GSYİH'yi elde etmek için nüfusla çarpmamız gerekir — bu tam olarak mutate()'in işidir.

```
gapminder |>
  filter(year == 2007, country == "Turkey") |>
  mutate(toplam_gsyih_milyar = gdpPercap * pop / 1e9) |>
  select(country, gdpPercap, pop, toplam_gsyih_milyar)
```

country	gdpPercap	pop	toplam_gsyih_milyar
Turkey	8458.276	71158647	601.8795

Bir sütunun adını değiştirmek (değerini değil) için rename() kullanılır: rename(gapminder, yasam_beklentisi = lifeExp).

group_by() ve summarize() — Gruplara Göre Özetlemek

Tek başına summarize() tüm veriyi tek bir satıra indirir. group_by() ile birleştirildiğinde, özet her grup için ayrı ayrı hesaplanır — karşılaştırmalı analizin temel aracı.

```
gapminder |>
  filter(year == 2007) |>
  group_by(continent) |>
  summarise(
    ort_yasam = round(mean(lifeExp), 1),
    n_ulke = n()
  ) |>
  arrange(desc(ort_yasam))
```

continent	ort_yasam	n_ulke
Oceania	80.7	2
Europe	77.6	30
Americas	73.6	25
Asia	70.7	33
Africa	54.8	52

💡 Tavsiye

`group_by()` sonrası zincire devam ederken grup yapısının hâlâ aktif olduğunu unutmayın; `summarise()`'den sonra çıktı otomatik olarak bir grup daha “açılır” (son gruba göre). Beklenmedik davranışlardan kaçınmak için karmaşık zincirlerin sonuna `ungroup()` eklemek iyi bir alışkanlıktır.

Fiilleri `|>` ile Zincirlemek

Gerçek analiz sorularının çoğu, tek bir fiille değil, birkaçının sırayla uygulanmasıyla cevaplanır. Örnek soru: “2007’de, nüfusu 10 milyonun üzerinde olan Asya ülkeleri arasında yaşam beklentisi en yüksek beşi hangisi?”

```
gapminder |>
  filter(year == 2007, continent == "Asia", pop > 10e6) |>
  arrange(desc(lifeExp)) |>
  select(country, lifeExp, pop) |>
  head(5)
```

country	lifeExp	pop
Japan	82.603	127467972
Korea, Rep.	78.623	49044790
Taiwan	78.400	23174294
Vietnam	74.249	85262356
Malaysia	74.241	24821286

Beş satırlık bu zincir, tek bir SQL sorgusu veya iç içe geçmiş beş fonksiyon çağrısı yerine, yukarıdan aşağıya **okunabilir bir cümle** gibi çalışır: filtrele → sırala → seç → ilk beşi al.

Düzenli (Tidy) Veri

Bir veri setinin “düzenli” (tidy) sayılması için üç kural sağlanmalıdır:

1. Her **değişken** kendi sütununu oluşturur.
2. Her **gözlem** kendi satırını oluşturur.
3. Her **değer** kendi hücreğini oluşturur.

gapminder zaten bu kurallara uyar: her satır bir ülke-yıl gözlemidir, her sütun bir değişkendir. Ama araştırmacılar veriyi genellikle bu biçimde **almazlar**. Gapminder Vakfı'nın kendi web sitesinden indirilen gösterge dosyaları, örneğin, her ülkeyi bir satıra, her **yılı ayrı bir sütuna** yerleştirir — insan gözüyle okumak için pratik, ama R'da filtrelemek veya gruplamak için elverişsizdir.

```
genis <- read_csv(here("data", "gapminder_genis_yasam_beklentisi.csv"))
```

```
Rows: 142 Columns: 14
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (2): country, continent
```

```
dbl (12): 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, 2002, ...
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
genis |> select(1:5) |> head(3)
```

country	continent	1952	1957	1962
Algeria	Africa	43.077	45.685	48.303
Angola	Africa	30.015	31.999	34.000
Benin	Africa	38.223	40.358	42.618

Bu tablo düzenli değildir: “yıl” bir değişkendir ama burada 12 ayrı sütuna (1952, 1957, ... 2007) yayılmıştır. `filter(year == 1997)` gibi bir komut bu hâliyle **çalışmaz** — çünkü `year` diye bir sütun yok.

`pivot_longer()` — Genişten Uzuna

```
uzun <- genis |>
  pivot_longer(
    cols = `1952`:`2007`,
    names_to = "yil",
    values_to = "yasam_beklentisi"
  ) |>
  mutate(yil = as.integer(yil))

uzun |> head(4)
```

country	continent	yıl	yasam_beklentisi
Algeria	Africa	1952	43.077
Algeria	Africa	1957	45.685
Algeria	Africa	1962	48.303
Algeria	Africa	1967	51.407

142 satırlık geniş tablo, 1704 satırlık uzun bir tabloya dönüştü (142×12 yıl). Artık `filter(yıl == 1997)` veya `group_by(continent) |> summarise(...)` gibi tüm dplyr fiilleri sorunsuz çalışır.

i Not

İsim ile değer arasındaki fark

`names_to` argümanı, eski **sütun adlarının** ("1952", "1957", ...) hangi yeni sütuna gireceğini belirtir. `values_to` argümanı ise, o sütunlardaki **hücre değerlerinin** hangi yeni sütuna gireceğini belirtir. Karıştırmamanın en yaygın belirtisi: `yıl` sütununda yaşam beklentisi değerlerinin, `yasam_beklentisi` sütununda ise yılların görünmesidir.

pivot_wider() — Uzun dan Geniş e

Tersi yön de gereklidir — özellikle bir **rapor** hazırlarken. Diyelim ki bir politika notu için kıtaların yıllara göre ortalama yaşam beklentisini tek bir tabloda göstermek istiyorsunuz:

```
rapor_tablosu <- gapminder |>
  filter(year %in% c(1987, 1997, 2007)) |>
  group_by(continent, year) |>
  summarise(ort_yasam = round(mean(lifeExp), 1), .groups = "drop") |>
  pivot_wider(names_from = year, values_from = ort_yasam)

rapor_tablosu
```

continent	1987	1997	2007
Africa	53.3	53.6	54.8
Americas	68.1	71.2	73.6
Asia	64.9	68.0	70.7
Europe	73.6	75.5	77.6
Oceania	75.3	78.2	80.7

`pivot_wider()`, `names_from` argümanındaki sütunun **değerlerini** yeni sütun adlarına, `values_from` argümanındaki sütunun değerlerini de bu yeni sütunların içeriğine dönüştürür — `pivot_longer()`'ın tam tersi.

💡 Tavsiye

Genel kural: **analiz için uzun, sunum için geniş**. `dplyr` fiillerinin hepsi uzun formatta rahat çalışır; ama bir tabloyu Word'e veya bir slayta koyacaksanız, geniş format genellikle daha okunaklıdır.

İlişkisel Veri: Tabloları Birleştirmek

`gapminder` tek başına bir soruya cevap veremez: “**Bir ülkenin bölgesel bir örgüte üye olması, ekonomik göstergeleriyle ilişkili mi?**” Bu soruyu cevaplamak için ikinci bir tabloya ihtiyacımız var — üye ülke-örgüt eşleşmesini tutan bir tablo.

```
orgutler <- read_csv(here("data", "orgutler.csv"))
```

```
Rows: 4 Columns: 3
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (2): orgut_kodu, orgut_adi
```

```
dbl (1): kurulus_yili
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
uyelikler <- read_csv(here("data", "orgut_uyelikleri.csv"))
```

```
Rows: 60 Columns: 2
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (2): ulke, orgut_kodu
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
orgutler
```

orgut_kodu	orgut_adi	kurulus_yili
AB	Avrupa Birliđi	1993
NATO	Kuzey Atlantik Antlaşması Örgütü	1949
ASEAN	Güneydođu Asya Uluslar Birliđi	1967
MERCOSUR	Güney Ortak Pazarı	1991

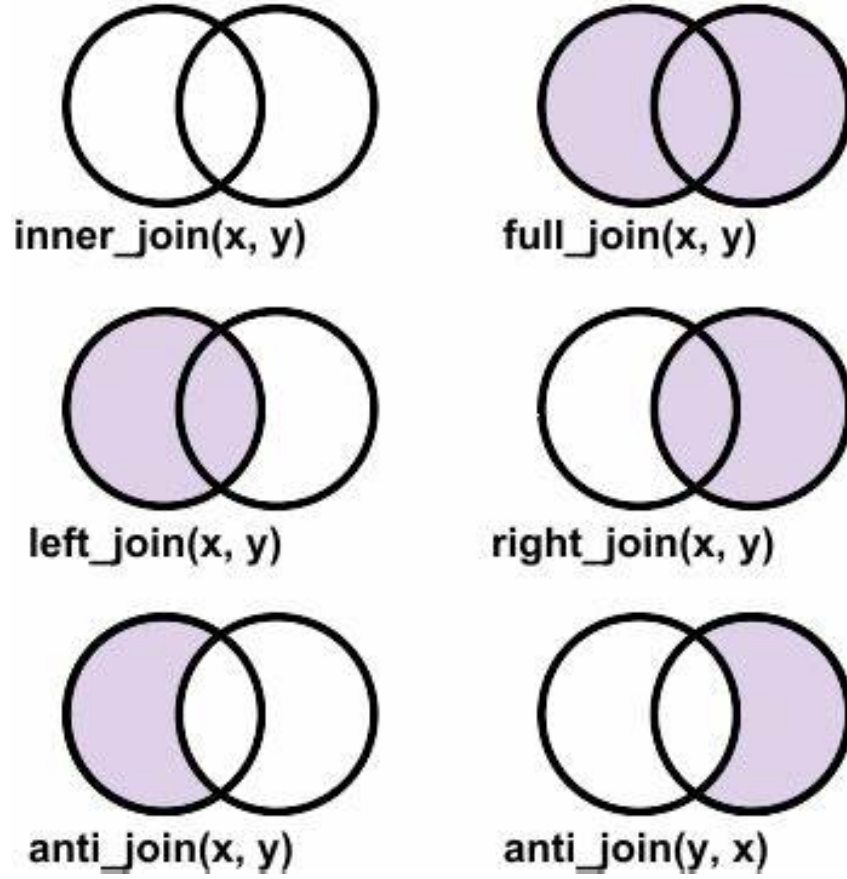
```
uyelikler |> head(4)
```

ulke	orgut_kodu
Austria	AB
Belgium	AB
Bulgaria	AB
Croatia	AB

İki tablo ortak bir sütunla bağlanır: `orgutler$orgut_kodu` ile `uyelikler$orgut_kodu`. `gapminder$country` ile `uyelikler$ulke` de aynı şekilde bağlanabilir — sütun adları farklı olsa bile.

Birleştirme (join) fiillerinin nasıl davrandığını gösteren şema:

```
knitr::include_graphics(here("05_modul", "images", "joins.png"))
```



Figür 2: Birleştirme (join) türleri

`inner_join()` — Sadece Eşleşenler

`inner_join()`, her iki tabloda da karşılığı olan satırları tutar.

```
uyelikler |> inner_join(orgutler, by = "orgut_kodu") |> head(4)
```

ulke	orgut_kodu	orgut_adi	kurulus_yili
Austria	AB	Avrupa Birliği	1993
Belgium	AB	Avrupa Birliği	1993
Bulgaria	AB	Avrupa Birliği	1993
Croatia	AB	Avrupa Birliği	1993

`gapminder`'ı üyelik tablosuyla birleştirdiğimizde, yalnızca en az bir örgüte üye olan ülkeler kalır:

```
gm2007 <- gapminder |> filter(year == 2007)

gm2007_orgut <- gm2007 |>
  inner_join(uyelikler, by = c("country" = "ulke"))

nrow(gm2007)
```

```
[1] 142
```

```
nrow(gm2007_orgut)
```

```
[1] 60
```

142 ülkeden 60 satıra düştük — çünkü bazı ülkeler birden fazla örgüte üye olduğu için **birden fazla satırda** görünüyor (ör. Belçika hem AB hem NATO üyesi), bazıları da hiçbirine üye değil.

Dış Birleştirmeler: left_join()

`inner_join()`'in aksine, `left_join(x, y)` **x'teki tüm satırları** korur; y'de eşleşme yoksa, o sütunlar NA ile doldurulur. Örgüt tablosundaki tam adı, üyelik tablosuna kaybetmeden eklemek için:

```
gm2007_orgut |>
  left_join(orgutler, by = "orgut_kodu") |>
  select(country, orgut_adi, gdpPercap) |>
  head(4)
```

country	orgut_adi	gdpPercap
Albania	Kuzey Atlantik Antlaşması Örgütü	5937.03
Argentina	Güney Ortak Pazarı	12779.38
Austria	Avrupa Birliği	36126.49
Belgium	Avrupa Birliği	33692.61

Filtreleme Birleştirmeleri: `semi_join()` ve `anti_join()`

Bu ikisi diğerlerinden farklıdır: **yeni sütun eklemesler**, yalnızca **x**'in satırlarını **y**'ye göre süzerler.

- `semi_join(x, y)`: **y**'de eşleşmesi olan **x** satırlarını **tutar**.
- `anti_join(x, y)`: **y**'de eşleşmesi olan **x** satırlarını **atar** (yani yalnızca eşleşmeyenler kalır).

```
ab_uyeleri <- uyelikler |> filter(orgut_kodu == "AB")

gm2007 |>
  semi_join(ab_uyeleri, by = c("country" = "ulke")) |>
  nrow()
```

[1] 21

`anti_join()`, veri kümenizin **kapsamındaki boşlukları** görmenin en hızlı yoludur. Örneğin, `gapminder`'daki Avrupa ülkelerinden hangileri bu dört örgütün (AB, NATO, ASEAN, MERCOSUR) hiçbirine üye değil?

```
gm2007 |>
  filter(continent == "Europe") |>
  anti_join(uyelikler, by = c("country" = "ulke")) |>
  select(country)
```

country
Bosnia and Herzegovina
Serbia
Switzerland

Sonuç mantıklı: İsviçre tarihsel tarafsızlığı nedeniyle her ikisinin de dışında, Bosna-Hersek ve Sırbistan ise ne AB ne NATO üyesi. Bu üçü dışındaki tüm Avrupa ülkeleri en az bir örgüte üye.

i Not

Veri kapsamı her zaman tamdır varsayımı tehlikelidir

Dikkat: bu üyelik tablosu, AB'nin gerçek 27 üyesinin yalnızca 21'ini içeriyor — çünkü `gapminder`'ın 142 ülkelik paneli Kıbrıs, Estonya, Letonya, Litvanya, Lüksemburg ve

Malta gibi küçük ülkeleri kapsamıyor. `anti_join()` size veri setiniz **içindeki** boşlukları gösterir; ama veri setinizin kendisinin dış dünyayı ne kadar eksiksiz temsil ettiğini göstermez. İkisini birbirine karıştırmayın.

Hepsini Bir Arada: Örgüt Üyeliğine Göre Karşılaştırma

Şimdi `select()`, `filter()`, `mutate()`, `group_by()`, `summarize()` ve `left_join()`'i tek bir zincirde birleştirerek asıl soruyu cevaplayalım:

```
gm2007 |>
  inner_join(uyelikler, by = c("country" = "ulke")) |>
  left_join(orgutler, by = "orgut_kodu") |>
  group_by(orgut_adi) |>
  summarise(
    n_ulke = n(),
    ort_gdp = round(mean(gdpPercap)),
    ort_yasam = round(mean(lifeExp), 1),
    .groups = "drop"
  ) |>
  arrange(desc(ort_gdp))
```

orgut_adi	n_ulke	ort_gdp	ort_yasam
Avrupa Birliği	21	26405	77.8
Kuzey Atlantik Antlaşması Örgütü	27	25902	77.7
Güneydoğu Asya Uluslar Birliği	8	9860	70.4
Güney Ortak Pazarı	4	9157	74.0

Tek bir zincirde: iki tabloyu birleştirdik, örgüt adlarını ekledik, örgüte göre gruplandık ve üç özet istatistik hesapladık. Sonuç, AB ve NATO üyesi ülkelerin (örtüşen üyelikleriyle) ortalama kişi başı GSYİH'sinin ASEAN ve MERCOSUR'a göre belirgin biçimde yüksek olduğunu gösteriyor — ama bunun **nedeni** hakkında bu tablo tek başına hiçbir şey söylemez; korelasyon ile nedensellik arasındaki farkı bir sonraki modülde tekrar hatırlayacağız.

Özet

Not

Bu modülde öğrendikleriniz

- `select()`, `filter()`, `mutate()`, `arrange()`, `group_by()+summarize()` — dplyr'ın beş temel fiili
- `|>` ile fiilleri zincirlemek
- Düzenli (tidy) verinin üç kuralı
- `pivot_longer()` ve `pivot_wider()` ile geniş/uzun format arası dönüşüm
- `inner_join()`, `left_join()`, `semi_join()`, `anti_join()` ile birden çok tabloyu birleştirmek

Sırada Ne Var?

Bir sonraki oturumda bu modülle birlikte işlenen **Quarto ile raporlama** konusuna bakacağız (06_modul/06_modul_ders_quarto.qmd) — bugün ürettiğiniz tabloları, kod ve düz metinle birlikte tek bir belgede nasıl yayınlanabilir hâle getireceğinizi öğreneceksiniz. Ardından olasılık ve dağılımlarla çıkarımsal istatistiğe geçeceğiz.